Mathematical Specification of the QuantShield Actor-Critic Framework

Author: Calvin J. Lomax

Abstract

This document states the mathematical objects, objectives, and evaluation criteria underlying the QuantShield transformer actor-critic framework. The system is an offline reinforcement-learning model trained from historical portfolio-weight demonstrations and realized forward returns. The actor maps a multi-asset return state into a long-only portfolio distribution, while the critic estimates the value of a state-action pair. Training combines behavior cloning, critic regression, entropy regularization, and reward maximization against benchmark, equal-weight, restricted-random, and Markowitz baselines. The framework uses modern portfolio theory, actor-critic reinforcement learning, offline RL, transformer attention, Dirichlet policies, and adaptive stochastic optimization [1]-[9].

1. Market and Portfolio Notation

Let the investable universe contain N assets indexed by $i \in \{1, \dots, N\}$. Let $p_{i,t}$ denote the adjusted price of asset i at time t. The simple return is

$$r_{i,t} = \frac{p_{i,t}}{p_{i,t-1}} - 1.$$

The return vector at time t is

$$\mathbf{r}_t = \begin{bmatrix} r_{1,t} & r_{2,t} & \cdots & r_{N,t} \end{bmatrix}^\top \in \mathbb{R}^N.$$

A portfolio action is a long-only fully invested weight vector

$$\mathbf{w}_t = \begin{bmatrix} w_{1,t} & w_{2,t} & \cdots & w_{N,t} \end{bmatrix}^\top \in \Delta^{N-1},$$

where the unit simplex is

$$\Delta^{N-1} = \left\{ \mathbf{w} \in \mathbb{R}^N : \; w_i \geq 0, \; \sum_{i=1}^{N} w_i = 1 \right\}.$$

For a forward segment $F_t = \{t+1, \dots, t+H\}$, the portfolio cumulative return under weights $\mathbf{w}_t$ is

$$R(\mathbf{w}_t; F_t) = \prod_{\tau=t+1}^{t+H} \left(1 + \mathbf{r}_\tau^\top \mathbf{w}_t\right) - 1.$$

This expression corresponds to the implementation-level segment return:

$$\texttt{daily\_returns} = \mathbf{R}_{F_t} \mathbf{w}_t, \qquad R = \prod_{\tau} (1 + \texttt{daily\_returns}_\tau) - 1.$$

2. Offline State Construction

At each rebalance date t, the model consumes a lookback window of length L. The raw window is

$$\mathbf{X}_t = \begin{bmatrix} \mathbf{r}_{t-L+1}^\top \\ \mathbf{r}_{t-L+2}^\top \\ \vdots \\ \mathbf{r}_t^\top \end{bmatrix} \in \mathbb{R}^{L \times N}.$$

QuantShield forms three features per asset and date:

1. raw return,
2. volatility-normalized return,
3. cumulative lookback return.

For each asset i, define the lookback volatility estimate

$$\hat{\sigma}_{i,t} = \mathrm{Std}\left(r_{i,t-L+1}, \ldots, r_{i,t}\right) + \varepsilon, \qquad \varepsilon > 0.$$

The normalized return is

$$z_{i,\tau} = \frac{r_{i,\tau}}{\hat{\sigma}_{i,t}}, \qquad \tau \in \{t-L+1, \ldots, t\}.$$

The cumulative feature is

$$c_{i,\tau} = \sum_{u=t-L+1}^{\tau} r_{i,u}.$$

The resulting state tensor is

$$\mathbf{S}_t \in \mathbb{R}^{N \times L \times 3}, \qquad \mathbf{S}_{t,i,\ell} = \begin{bmatrix} r_{i,t-L+\ell} \\ z_{i,t-L+\ell} \\ c_{i,t-L+\ell} \end{bmatrix}.$$

3. Demonstration Actions

The offline dataset includes demonstrated portfolio actions

$$\mathbf{a}_t^{\mathrm{demo}} \in \Delta^{N-1}.$$

These demonstrations are generated from saved optimization suites or portfolio-fit objectives. If a raw action vector $\tilde{\mathbf{a}}$ is not exactly on the simplex, QuantShield normalizes it as

$$\mathcal{N}_\Delta(\tilde{\mathbf{a}})_i = \frac{\max(\tilde{a}_i, \varepsilon)}{\sum_{j=1}^{N} \max(\tilde{a}_j, \varepsilon)}.$$

The raw demonstration return is

$$R_t^{\mathrm{demo}} = R(\mathbf{a}_t^{\mathrm{demo}}; F_t).$$

4. Baseline Returns

QuantShield compares the policy against several baselines. These baselines enter both reward construction and evaluation.

4.1 Benchmark Ticker

For a benchmark asset b, the benchmark return over the forward segment is

$$R_t^{\text{bench}} = \prod_{\tau=t+1}^{t+H} (1 + r_{b,\tau}) - 1.$$

4.2 Equal-Weight Portfolio

The equal-weight portfolio is a naive diversification baseline [10]:

$$\mathbf{w}^{\text{eq}} = \frac{1}{N}\mathbf{1}.$$

Its forward return is

$$R_t^{\text{eq}} = R(\mathbf{w}^{\text{eq}}; F_t) = \prod_{\tau=t+1}^{t+H} \left(1 + \frac{1}{N}\sum_{i=1}^{N} r_{i,\tau}\right) - 1.$$

4.3 Restricted-Random Portfolio

The restricted-random baseline samples a feasible random allocation subject to lower and upper bounds:

$$\mathbf{w}^{\text{rr}} \in \left\{\mathbf{w} \in \Delta^{N-1} : \; w_{\min} \le w_i \le w_{\max}\right\}.$$

In the implementation, a Dirichlet sample is clipped and renormalized if necessary. Its realized return is

$$R_t^{\text{rr}} = R(\mathbf{w}^{\text{rr}}; F_t).$$

4.4 Markowitz Mean-Variance Portfolio

Let $\boldsymbol{\mu}_t$ and $\boldsymbol{\Sigma}_t$ be the estimated annualized mean vector and covariance matrix from the lookback window. QuantShield may use covariance shrinkage to improve the conditioning of $\boldsymbol{\Sigma}_t$ [2]. The long-only Markowitz baseline solves the mean-variance allocation problem [1]:

$$\mathbf{w}_t^{\text{mv}} = \arg\max_{\mathbf{w}\in\Delta^{N-1}} \left[\boldsymbol{\mu}_t^\top \mathbf{w} - \frac{\lambda}{2}\mathbf{w}^\top \boldsymbol{\Sigma}_t \mathbf{w}\right],$$

subject to the additional cap

$$0 \le w_i \le w_{\max}.$$

The Markowitz forward return is

$$R_t^{\mathrm{mv}} = R(\mathbf{w}_t^{\mathrm{mv}}; F_t).$$

If the covariance estimate is not usable, the implementation falls back to equal weights:

$$\mathbf{w}_t^{\mathrm{mv}} \leftarrow \mathbf{w}^{\mathrm{eq}}.$$

5. Composite Training Reward

Let the policy or demonstration raw return be

$$R_t^{\mathrm{raw}}.$$

The separate-baseline reward mode uses

$$\mathcal{R}_t^{\mathrm{separate}} = \bar{w}_0 R_t^{\mathrm{raw}} + \bar{w}_b(R_t^{\mathrm{raw}} - R_t^{\mathrm{bench}}) + \bar{w}_e(R_t^{\mathrm{raw}} - R_t^{\mathrm{eq}}) + \bar{w}_r(R_t^{\mathrm{raw}} - R_t^{\mathrm{rr}}) + \bar{w}_m(R_t^{\mathrm{raw}} - R_t^{\mathrm{mv}}).$$

The normalized weights are

$$\bar{w}_k = \frac{w_k}{\sum_j |w_j|}.$$

The default New Model reward mode compares the policy against the best selected deterministic baseline:

$$R_t^{\mathrm{best}} = \max\left(R_t^{\mathrm{bench}}, R_t^{\mathrm{eq}}, R_t^{\mathrm{mv}}\right).$$

With

$$w_{\mathrm{cmp}} = w_b + w_e + w_m,$$

the best-of-selected reward is

$$\mathcal{R}_t^{\mathrm{best}} = \bar{w}_0 R_t^{\mathrm{raw}} + \bar{w}_{\mathrm{cmp}}\left(R_t^{\mathrm{raw}} - R_t^{\mathrm{best}}\right) + \bar{w}_r\left(R_t^{\mathrm{raw}} - R_t^{\mathrm{rr}}\right),$$

where

$$\bar{w}_0, \bar{w}_{\mathrm{cmp}}, \bar{w}_r = \frac{(w_0, w_{\mathrm{cmp}}, w_r)}{|w_0| + |w_{\mathrm{cmp}}| + |w_r|}.$$

This reward design encourages the model to exceed the strongest of benchmark, equal-weight, and Markowitz for each sample, while still considering restricted-random robustness.

6. Actor-Critic Model

6.1 Per-Asset Projection

For each asset i, flatten its lookback state:

$$\mathbf{x}_{i,t} = \mathrm{vec}\left(\mathbf{S}_{t,i,:,:}\right) \in \mathbb{R}^{3L}.$$

The input projection is

$$\mathbf{h}_{i,t}^{(0)} = \mathrm{GELU}\left(\mathrm{LayerNorm}\left(\mathbf{W}_p \mathbf{x}_{i,t} + \mathbf{b}_p\right)\right) + \mathbf{e}_i,$$

where $\mathbf{e}_i \in \mathbb{R}^d$ is the learned asset embedding.

6.2 Cross-Asset Transformer Encoder

Let

$$\mathbf{H}_t^{(0)} = \begin{bmatrix} (\mathbf{h}_{1,t}^{(0)})^\top \\ \vdots \\ (\mathbf{h}_{N,t}^{(0)})^\top \end{bmatrix} \in \mathbb{R}^{N \times d}.$$

The transformer encoder maps the asset-token matrix through self-attention layers [7]:

$$\mathbf{H}_t = f_\theta^{\mathrm{enc}}\left(\mathbf{H}_t^{(0)}\right) \in \mathbb{R}^{N \times d}.$$

For a single attention head, the attention operation is

$$\mathrm{Attn}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \mathrm{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}.$$

Multi-head self-attention concatenates M such heads:

$$\mathrm{MHA}(\mathbf{H}) = \mathrm{Concat}\left(\mathrm{head}_1, \ldots, \mathrm{head}_M\right)\mathbf{W}^O.$$

This allows each asset representation to condition on the other assets in the candidate portfolio.

7. Actor: Dirichlet Policy

The actor head maps each encoded asset vector to a scalar logit:

$$z_{i,t} = f_\theta^{\mathrm{actor}}\left(\mathbf{H}_{t,i}\right).$$

The Dirichlet concentration parameter is

$$\alpha_{i,t} = \mathrm{softplus}(z_{i,t}) + 1.$$

Thus

$$\boldsymbol{\alpha}_t = \begin{bmatrix} \alpha_{1,t} & \cdots & \alpha_{N,t} \end{bmatrix}^\top .$$

The stochastic policy is a Dirichlet distribution over the portfolio simplex [6]:

$$\pi_\theta(\mathbf{a}_t \mid \mathbf{S}_t) = \text{Dirichlet}\left(\boldsymbol{\alpha}_t\right), \qquad \mathbf{a}_t \in \Delta^{N-1}.$$

The deterministic inference policy is the Dirichlet mean:

$$\hat{\mathbf{w}}_t = \mathbb{E}_{\pi_\theta}\left[\mathbf{a}_t \mid \mathbf{S}_t\right] = \frac{\boldsymbol{\alpha}_t}{\sum_{j=1}^{N} \alpha_{j,t}}.$$

This construction enforces nonnegative weights that sum to one.

8. Critic: State-Action Value Function

The encoded context is pooled across assets:

$$\bar{\mathbf{h}}_t = \frac{1}{N} \sum_{i=1}^{N} \mathbf{H}_{t,i}.$$

For an action $\mathbf{a}_t$, the critic input is

$$\mathbf{u}_t = \begin{bmatrix} \bar{\mathbf{h}}_t \\ \mathbf{a}_t \end{bmatrix}.$$

The critic predicts the scalar value

$$Q_\phi(\mathbf{S}_t, \mathbf{a}_t) = f_\phi^{\text{critic}}\left(\mathbf{u}_t\right).$$

During training, the critic is evaluated on both the demonstration action and the actor mean action:

$$Q_\phi^{\text{demo}} = Q_\phi(\mathbf{S}_t, \mathbf{a}_t^{\text{demo}}),$$

$$Q_\phi^{\text{policy}} = Q_\phi(\mathbf{S}_t, \hat{\mathbf{w}}_t).$$

9. Training Losses

Let the normalized training reward target be

$$\tilde{\mathcal{R}}_t = \frac{\mathcal{R}_t - \mu_{\mathcal{R}}}{\sigma_{\mathcal{R}} + \varepsilon}.$$

The critic loss is mean-squared error on demonstration actions:

$$\mathcal{L}_{\text{critic}} = \frac{1}{B} \sum_{t \in \mathcal{B}} \left(Q_\phi(\mathbf{S}_t, \mathbf{a}_t^{\text{demo}}) - \tilde{\mathcal{R}}_t\right)^2.$$

The behavior-cloning loss follows imitation-learning practice in which the policy is anchored to demonstrated actions [11]:

$$\mathcal{L}_{\mathrm{BC}} = \frac{1}{B} \sum_{t \in \mathcal{B}} \left\| \hat{\mathbf{w}}_t - \mathbf{a}_t^{\mathrm{demo}} \right\|_2^2 .$$

The Dirichlet entropy is

$$\mathcal{H}\left[\pi_\theta(\cdot \mid \mathbf{S}_t)\right] = \log B(\boldsymbol{\alpha}_t) + (\alpha_{0,t} - N)\psi(\alpha_{0,t}) - \sum_{i=1}^{N} (\alpha_{i,t} - 1)\psi(\alpha_{i,t}),$$

where

$$\alpha_{0,t} = \sum_{i=1}^{N} \alpha_{i,t}, \qquad B(\boldsymbol{\alpha}) = \frac{\prod_i \Gamma(\alpha_i)}{\Gamma(\sum_i \alpha_i)}.$$

The actor loss is

$$\mathcal{L}_{\mathrm{actor}} = -\frac{1}{B} \sum_{t \in \mathcal{B}} Q_\phi(\mathbf{S}_t, \hat{\mathbf{w}}_t) + \lambda_{\mathrm{BC}} \mathcal{L}_{\mathrm{BC}} - \lambda_{\mathrm{H}} \frac{1}{B} \sum_{t \in \mathcal{B}} \mathcal{H}\left[\pi_\theta(\cdot \mid \mathbf{S}_t)\right].$$

The total optimized loss is

$$\mathcal{L} = \mathcal{L}_{\mathrm{actor}} + \mathcal{L}_{\mathrm{critic}}.$$

Gradients are clipped:

$$\nabla \mathcal{L} \leftarrow \nabla \mathcal{L} \cdot \min\left(1, \frac{c}{\|\nabla \mathcal{L}\|_2}\right),$$

where c is the configured gradient-clip norm.

Parameters are updated with Adam or AdamW [8], [9] in PyTorch [14]:

$$\theta, \phi \leftarrow \mathrm{OptimizerStep}\left(\theta, \phi, \nabla_{\theta,\phi} \mathcal{L}\right).$$

10. Validation and Model Selection

The offline dataset is split chronologically:

$$\mathcal{D} = \mathcal{D}_{\mathrm{train}} \cup \mathcal{D}_{\mathrm{val}}.$$

For a validation sample, the policy action is

$$\hat{\mathbf{w}}_t = \frac{\boldsymbol{\alpha}_t}{\sum_j \alpha_{j,t}},$$

and the realized policy return is

$$R_t^{\mathrm{policy}} = R(\hat{\mathbf{w}}_t; F_t).$$

The benchmark excess return is

$$E_t^{\text{bench}} = R_t^{\text{policy}} - R_t^{\text{bench}}.$$

Similarly,

$$E_t^{\text{eq}} = R_t^{\text{policy}} - R_t^{\text{eq}},$$

$$E_t^{\text{rr}} = R_t^{\text{policy}} - R_t^{\text{rr}},$$

$$E_t^{\text{mv}} = R_t^{\text{policy}} - R_t^{\text{mv}}.$$

QuantShield ranks candidate checkpoints by multi-baseline evidence. For a generic excess series $\{E_t\}_{t=1}^n$, one-sided outperformance is summarized with a Student-style t-statistic [12]. Define

$$\bar{E} = \frac{1}{n}\sum_{t=1}^{n} E_t,$$

$$s_E^2 = \frac{1}{n-1}\sum_{t=1}^{n}(E_t - \bar{E})^2,$$

$$t_E = \frac{\bar{E}}{s_E/\sqrt{n}}.$$

The one-sided outperformance test is

$$H_0 : \mathbb{E}[E] \le 0, \qquad H_1 : \mathbb{E}[E] > 0.$$

The model-selection key prioritizes:

1. number of baselines significantly beaten,
2. average excess return across baselines,
3. Markowitz excess,
4. benchmark excess,
5. average t-statistic.

In symbolic form, a candidate is ranked by

$$\text{ScoreKey} = \left(S, \bar{E}_{\text{avg}}, \bar{E}_{\text{mv}}, \bar{E}_{\text{bench}}, \bar{t}_{\text{avg}}\right),$$

where

$$S = \sum_{k\in\{\text{bench,eq,rr,mv}\}} \mathbf{1}\left[p_k < 0.05 \wedge \bar{E}_k > 0\right],$$

and

$$\bar{E}_{\text{avg}} = \frac{1}{4}\left(\bar{E}_{\text{bench}} + \bar{E}_{\text{eq}} + \bar{E}_{\text{rr}} + \bar{E}_{\text{mv}}\right).$$

11. Inference

At inference time, the system does not use the critic. Given a fresh state $\mathbf{S}_t$, the encoder and actor produce $\boldsymbol{\alpha}_t$, and the allocation is

$$\mathbf{w}_t^{\text{model}} = \frac{\boldsymbol{\alpha}_t}{\sum_{j=1}^{N} \alpha_{j,t}}.$$

If integer-share execution is required for a capital base C_t, with asset prices $\mathbf{p}_t$, the desired dollar allocation is

$$\mathbf{d}_t = C_t \mathbf{w}_t^{\text{model}}.$$

The initial integer share estimate is

$$\mathbf{q}_t = \left\lfloor \frac{\mathbf{d}_t}{\mathbf{p}_t} \right\rfloor.$$

Residual cash is

$$\text{cash}_t = C_t - \sum_{i=1}^{N} q_{i,t} p_{i,t}.$$

The executed portfolio value at the next date is

$$V_{t+1} = \sum_{i=1}^{N} q_{i,t} p_{i,t+1} + \text{cash}_t.$$

12. Replay Metrics

Let V_t^{policy} be the model portfolio value and V_t^{bench} the benchmark portfolio value. The cumulative return through time T is

$$\text{CR}_T = \frac{V_T}{V_0} - 1.$$

The per-period portfolio return is

$$\rho_t = \frac{V_t}{V_{t-1}} - 1.$$

Annualized return for P periods per year is

$$\text{AnnRet} = \left(\prod_{t=1}^{T} (1 + \rho_t) \right)^{P/T} - 1.$$

Annualized volatility is

$$\text{AnnVol} = \sqrt{P}\, \text{Std}(\rho_t).$$

Given risk-free rate r_f, Sharpe ratio is [13]

$$\text{Sharpe} = \frac{\text{AnnRet} - r_f}{\text{AnnVol}}.$$

Drawdown is

$$D_t = \frac{V_t}{\max_{u \le t} V_u} - 1.$$

Maximum drawdown is

$$\text{MDD} = \min_t D_t.$$

Tracking error against benchmark returns ρ_t^{bench} is

$$\text{TE} = \sqrt{P}\,\text{Std}\left(\rho_t - \rho_t^{\text{bench}}\right).$$

Beta is

$$\beta = \frac{\text{Cov}\left(\rho_t, \rho_t^{\text{bench}}\right)}{\text{Var}\left(\rho_t^{\text{bench}}\right)}.$$

13. Conceptual Summary

The actor-critic framework can be summarized as the following map:

$$\mathbf{S}_t \xrightarrow{\text{projection + asset embedding}} \mathbf{H}_t^{(0)} \xrightarrow{\text{cross-asset transformer}} \mathbf{H}_t \xrightarrow{\text{actor}} \pi_\theta(\mathbf{a}_t \mid \mathbf{S}_t) \xrightarrow{\mathbb{E}[\cdot]} \hat{\mathbf{w}}_t.$$

Training augments this path with the critic:

$$(\mathbf{H}_t, \mathbf{a}_t) \xrightarrow{\text{critic}} Q_\phi(\mathbf{S}_t, \mathbf{a}_t),$$

and the reward system:

$$\hat{\mathbf{w}}_t \xrightarrow{\text{forward returns}} R_t^{\text{policy}} \xrightarrow{\text{baseline comparisons}} \mathcal{R}_t.$$

The result is an offline policy that learns from portfolio-optimization demonstrations while being directly evaluated against practical investment baselines.

References

[1] H. Markowitz, "Portfolio selection," *The Journal of Finance*, vol. 7, no. 1, pp. 77-91, Mar. 1952.

[2] O. Ledoit and M. Wolf, "A well-conditioned estimator for large-dimensional covariance matrices," *Journal of Multivariate Analysis*, vol. 88, no. 2, pp. 365-411, Feb. 2004.

[3] S. Maillard, T. Roncalli, and J. Teiletche, "The properties of equally weighted risk contribution portfolios," *The Journal of Portfolio Management*, vol. 36, no. 4, pp. 60-70, Summer 2010.

[4] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.

[5] S. Levine, A. Kumar, G. Tucker, and J. Fu, "Offline reinforcement learning: Tutorial, review, and perspectives on open problems," arXiv:2005.01643, 2020.

[6] T. P. Minka, "Estimating a Dirichlet distribution," Microsoft Research, Cambridge, U.K., Tech. Rep., 2000.

[7] A. Vaswani *et al.*, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[8] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learning Representations*, 2015.

[9] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. Int. Conf. Learning Representations*, 2019.

[10] V. DeMiguel, L. Garlappi, and R. Uppal, "Optimal versus naive diversification: How inefficient is the 1/N portfolio strategy?" *The Review of Financial Studies*, vol. 22, no. 5, pp. 1915-1953, May 2009.

[11] D. A. Pomerleau, "ALVINN: An autonomous land vehicle in a neural network," in *Advances in Neural Information Processing Systems*, vol. 1, 1989.

[12] Student, "The probable error of a mean," *Biometrika*, vol. 6, no. 1, pp. 1-25, Mar. 1908.

[13] W. F. Sharpe, "Mutual fund performance," *The Journal of Business*, vol. 39, no. 1, pp. 119-138, Jan. 1966.

[14] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems*, vol. 32, 2019.